DEVOPS WINTER ARC · 90 DAYS CHALLENGE

DAY 23 / 90

AWS Week — Day 1: IAM & Access Management

Level	Beginner	**Time**	2–3 hours (flexible)
Focus	Users · Groups · Roles · Policies · MFA · Least privilege	**Cost goal**	$0 planned AWS resource spend

Today's mission

In the uploaded Week 3 AWS guide, this is **Day 1**: IAM & Access Management. In the 90-day Winter Arc sequence, it is **Day 23**. Learn how AWS controls who can do what, secure your account, and practise the basics of users, groups, policies and roles.

Scope rule: Do not repeat Day 22's AWS overview/account setup or earlier Linux, Docker and Kubernetes tasks. Today focuses only on IAM and access management.

Before you begin — wallet and account safety

> **Important:** IAM itself is generally not the expensive part of AWS, but permissions can allow someone to create chargeable resources. Keep your $0 budget/alerts in place, enable MFA, never share credentials, and do not create access keys for practice users. Budget alerts are notifications—not a hard spending cap.

Part 1 — Learn the key concepts (20 minutes)

• **IAM user:** an identity for a person or workload; it may have long-term credentials.

• **Group:** a collection of IAM users. Attach common permissions to groups instead of managing each user separately.

• **Role:** an identity that is assumed to obtain temporary credentials; commonly used by AWS services and cross-account access.

• **Policy:** a JSON permissions document describing Effect, Action, Resource and optionally Condition.

• **Least privilege:** grant only the minimum permissions required for a task.

• **MFA:** multi-factor authentication adds another verification step.

• **Default deny:** requests are denied unless an applicable Allow exists; an explicit Deny overrides an Allow.

• **Trust policy:** defines who is allowed to assume a role.

Part 2 — Secure the account (15–20 minutes)

■ Sign in to the AWS console as the root user only to secure the account; enable MFA if you have not already.

■ Do not use the root user for daily tasks. Follow your organisation/account setup guidance to use a separate, appropriately permissioned identity.

■ Confirm your billing budget and alerts from Day 22 are still configured. Do not launch EC2, RDS, NAT Gateway, EKS or load balancers for this assignment.

■ Do not share screenshots containing account IDs, email addresses, QR codes, MFA recovery details, secret keys or passwords.

Part 3 — IAM console exploration (20 minutes)

Open the IAM console and inspect the Users, User groups, Roles, Policies and Account settings sections. Read only; do not attach broad permissions to your everyday identity.

■ Find where MFA status is shown.

■ Open the policy list and inspect the difference between AWS-managed and customer-managed policies.

■ Open one example policy and identify Effect, Action and Resource.

■ Find the role page and locate the trust relationship/trust policy area.

Part 4 — Practical lab: design access for a small team (30–45 minutes)

Scenario: A small company has three teams. Design access for Admins, Developers and Auditors. For this beginner session, you may complete the design on paper/JSON first. Only create IAM objects if your account is authorised for the lab and you understand the permissions.

Group	Intended access	Design note
Admins	Administrative access only for a designated admin identity	Keep membership limited; never use root for daily work.
Developers	Read-only baseline plus read/write access to one lab S3 bucket in a later lab	Custom policy should target one bucket only; no account-wide admin rights.
Auditors	Read-only access	Should inspect resources, not modify them.

Task A — Make a permission matrix. Write down what each group should be allowed and not allowed to do. Explain why Developers should not receive AdministratorAccess.

Task B — Read this sample policy. Identify the allowed action and resource. This is an example only; replace the bucket placeholder before using it and do not attach it unless you understand the scope.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::YOUR-LAB-BUCKET/*"
  }]
}
```

Note: object permissions above do not grant bucket listing. A separate s3:ListBucket permission uses the bucket ARN without /*. Avoid using real or production data in a practice bucket.

Task C — Understand role vs user. Write one example of when an EC2 instance should use a role instead of storing access keys on the server.

Part 5 — Optional CLI identity check (only if AWS CLI is already configured)

Do not create root access keys to make this work. If AWS CLI is not installed or configured, skip this section today; setup can be completed later using an appropriately permissioned non-root identity.

```
$ aws --version
$ aws sts get-caller-identity
$ aws configure list
$ aws iam get-account-summary
```

What these commands do

• **aws --version** checks whether the AWS CLI is installed.

• **aws sts get-caller-identity** shows the identity/account context currently being used. Verify you are in the intended account before running commands.

• **aws configure list** helps identify the active profile, Region and credential source; do not share its output publicly if it reveals sensitive context.

• **aws iam get-account-summary** reports selected IAM account statistics, subject to permissions.

Part 6 — Troubleshooting mini-challenge (10 minutes)

You can open IAM but cannot create a user. What might be wrong?

Suggested answer: Your identity may not have the required IAM permissions; read the AccessDenied message and ask the account owner/admin.

A Developer can read an object but cannot list the bucket. Why?

Suggested answer: s3:GetObject applies to object ARNs; listing requires s3:ListBucket on the bucket ARN.

A role cannot be assumed. What should you inspect?

Suggested answer: The role trust policy, caller permissions, target role ARN and any explicit Deny.

A policy contains Action: * and Resource: *. Is this least privilege?

Suggested answer: No. It grants an excessively broad scope and should be narrowed to required actions/resources.

Should an EC2 server store long-lived access keys in a file?

Suggested answer: No. Prefer an attached IAM role that supplies temporary credentials.

Interview questions

IAM user vs IAM role?

A user is an identity that can have long-term credentials; a role is assumed and provides temporary credentials.

What is least privilege?

Grant only the permissions required for the task.

How are IAM permissions evaluated?

Default deny; a matching Allow is needed, and an explicit Deny overrides an Allow.

Why use groups?

Permissions can be managed consistently for a team instead of one user at a time.

What is a trust policy?

It defines which principals may assume a role.

Why enable MFA on root?

It adds protection to the highest-privilege account identity.

Proof of work / screenshots

No GitHub submission is required for today. Capture only these important screenshots if you share progress on LinkedIn:

• IAM dashboard or MFA status (hide personal/account details).

• The sample policy JSON in your editor or your written permission matrix.

• If configured, the successful `aws sts get-caller-identity` output with account ID redacted.

Cleanup & completion checklist

■ Root MFA is enabled; root is not used for daily work.

■ I can explain user, group, role, policy and least privilege.

■ I designed Admins, Developers and Auditors permissions.

■ I understand why role credentials are preferred over long-lived access keys for AWS services.

■ I did not create chargeable resources or expose credentials.

Day 23 complete: IAM basics, secure account habits and a least-privilege access design. The source guide’s full Day 1 lab is a 3–4 hour task; this beginner-friendly assignment is a safe first step and can be expanded as you gain confidence.